

알고리즘

Chap 4. 탐욕 알고리즘 (Greedy Approach)

■ 탐욕 알고리즘 (Greedy Approach)

■ 탐욕 알고리즘

- 선택 시점에 가장 좋아 보이는 답을 선택함으로써 최종 답에 도달한다
- 어떤 선택이든지 선택할 당시에는(locally) 최적이지만, 전체적으로(globally) 최적인 해를 얻는다는 보장은 없다
- 선택된 결정은 되돌려지지 않는다.
 - 일단 어떤 동전을 선택하기로 하면 그 동전은 영원히 거스름 돈에 포함되고, 일단 어떤 동전을 포기하기로 하면 그 동전은 영원히 거스름돈에서 제외된다
- 탐욕 알고리즘은 최적의 해를 보장하지 않는다
 - 탐욕 알고리즘으로 설계했다면, 최적의 해를 주는지 항상 점검해야 한다

■ 탐욕 알고리즘 (Greedy Approach)

- (example) 거스름돈의 동전 개수를 최소로 만드는 문제

```
while (동전이 남아있고 문제 미해결) {  
    가장 가치가 높은 동전을 선택                // 선택과정(selection procedure)  
    if (동전을 추가하여 거스름 돈의 액수를 초과) // 적절성검사(feasibility check)  
        동전을 되돌려 놓는다  
    else  
        동전을 포함시킨다  
    if (동전을 추가하여 거스름 돈의 액수와 동일) // 해답점검(solution check)  
        문제 해결  
}
```

- (example) 거스름돈의 동전 개수를 최소로 만드는 문제

- 거스름 돈 액수: 36원
- 동전의 종류: 25, 10, 10, 5, 1, 1
- 1. 25원 선택 (총액) $25 < 36$
- 2. 10원 선택 (총액) $35 < 36$
- 3. 10원 선택, 복원 (총액) $45 > 36$
- 4. 5원 선택, 복원 (총액) $40 > 36$
- 5. 1원 선택 (총액) $36 == 36$

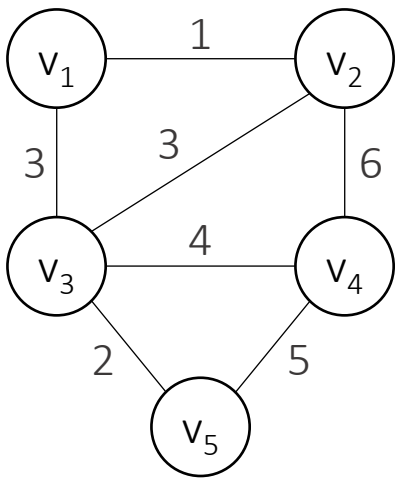
■ 탐욕 알고리즘 (Greedy Approach)

- 탐욕 알고리즘이 항상 최적의 해를 찾는다는 보장은 없다
 - (example) 12원 동전이 있는 경우, 최적해를 찾는다는 보장이 사라진다
 - 거스름 돈 액수: 16원
 - 동전의 종류: 12, 10, 5, 1, 1, 1, 1
 - 1. 12원 선택 (총액) $12 < 16$
 - 2. 10원 선택, 복원 (총액) $22 > 16$
 - 3. 5원 선택, 복원 (총액) $17 > 16$
 - 4. 1원 선택 (4번) (총액) $16 == 16$
 - 탐욕알고리즘의 결과는 5개 (12, 1, 1, 1, 1)
 - 최적의 해답은 3개 (10, 5, 1) : 탐욕 알고리즘은 최적해를 찾지 못한다.
- 탐욕 알고리즘의 절차
 - 공집합에서부터 문제의 해답이 될 때까지 원소를 반복적으로 추가
 - 선택과정(selection procedure): 현재 시점에서 최적을 선택하는 탐욕 기준에 따라서 집합에 추가할 다음 원소를 선택
 - 적절성검사(feasibility check): 업데이트된 집합이 해답이 되기에 적절한지 검사
 - 해답점검(solution check): 업데이트된 집합이 문제의 해답인지 결정

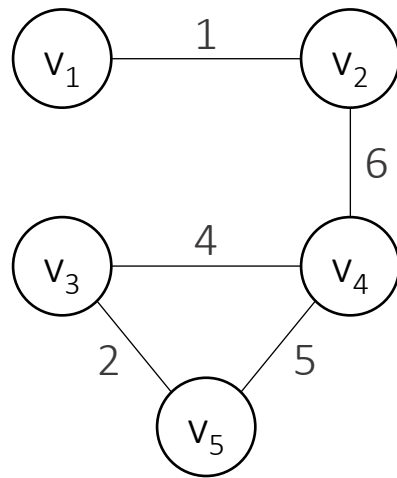
■ 최소비용 신장트리 (Minimum Spanning Trees)

■ 그래프 $G = (V, E)$

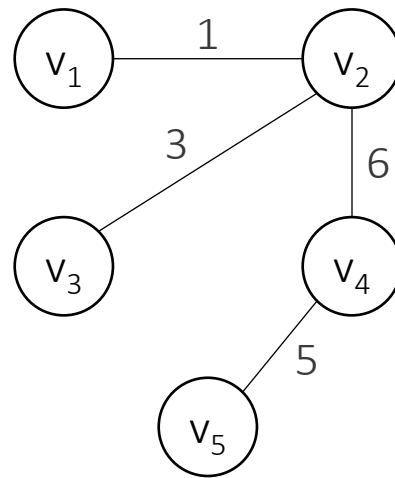
- (a)는 연결된 가중치 포함(weighted), 비방향(undirected) 그래프 G
- 비방향(undirected): edge에 방향이 없음
- Path: edge로 연결된 vertex의 배열
- 연결되어있다(connected): 비방향 그래프에서 모든 vertex의 쌍에 경로가 있음
- 단순순환경로(simple cycle): vertex에서 시작해서 다시 자신으로 돌아오는 경로
- 비순환적(acyclic): 단순순환경로가 없는 비방향 그래프 (ex. (c), (d))



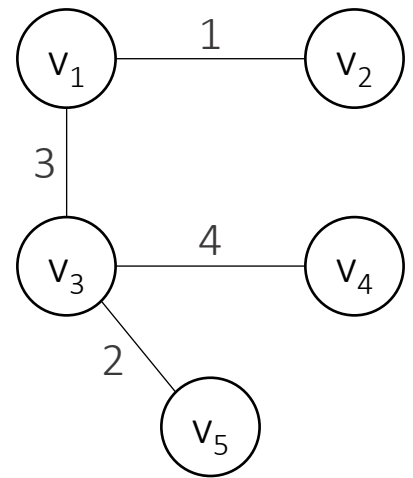
(a) connected, weighted, undirected graph G



(b) If (v_4, v_5) were removed from this subgraph, the graph would remain connected



(c) A spanning tree for G



(d) A minimum spanning tree for G

■ 최소비용 신장트리 (Minimum Spanning Trees)

■ Tree

- Free tree: 비순환적이며 연결되어 있는 비방향 그래프
- Rooted tree: vertex 하나를 root로 지정한 tree
- 신장트리(spanning tree): G 의 vertex를 모두 포함하면서 트리인 연결된 그래프
 - (c), (d)는 G 에 대한 신장트리
- 최소비용 신장트리(minimum spanning tree): 가중치의 합이 최소가 되는 신장트리
 - (d)는 G 에 대한 최소비용 신장트리

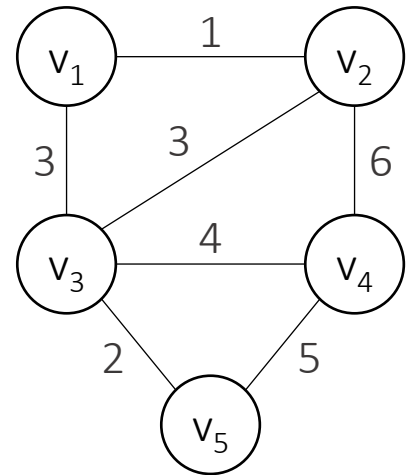
■ 최소비용 신장트리(minimum spanning tree) 문제

- 연결된 가중치 포함(weighted), 비방향(undirected) 그래프 G 에서 vertex는 모두 연결되어 있으면서도 edge의 가중치(weight)의 합이 최소가 되도록 edge를 제거하여 부분그래프를 만드는 문제
- 활용: 최소비용의 도로건설, 최소비용의 통신케이블 설치, 등
- 가중치의 합의 최소인 부분그래프는 반드시 트리가 된다
- 무작위 방법으로 찾으면, 최악의 경우 지수시간보다 나쁘다

■ 최소비용 신장트리 (Minimum Spanning Trees)

■ 그래프 $G = (V, E)$

- $V = \{v_1, v_2, v_3, v_4, v_5\}$
- $E = \{(v_1, v_2), (v_1, v_3), (v_2, v_3), (v_2, v_4), (v_3, v_4), (v_3, v_5), (v_4, v_5)\}$



■ 신장트리 $T = (V, F)$

- G 에 대한 최소비용 신장트리가 되도록 E 의 부분집합 F 찾기
- (high-level) 알고리즘 1

$F = \emptyset$

// edge의 집합을 공집합으로 초기화

while (미해결) {

 부분적으로 최적인 edge 선택

// 선택과정

 if (그 edge를 F 에 추가하면 순환경로가 생기지 않음)

// 적절성검사

 그 edge를 F 에 추가

 if (T 가 신장트리)

// 해답점검

 문제 해결

}

■ 프림(Prim)의 알고리즘 (Minimum spanning tree algorithm)

■ (high-level) 알고리즘 2

$F = \emptyset$

// edge의 집합을 공집합으로 초기화

$Y = \{v_1\}$

// vertex의 집합을 v_1 으로 초기화

while (미해결) {

Y 에서 가장 가까운 $V-Y$ 에 있는 vertex를 선택

// 선택과정 & 적절성검사

 그 vertex를 Y 에 추가

 그 edge를 F 에 추가

 if ($Y=V$)

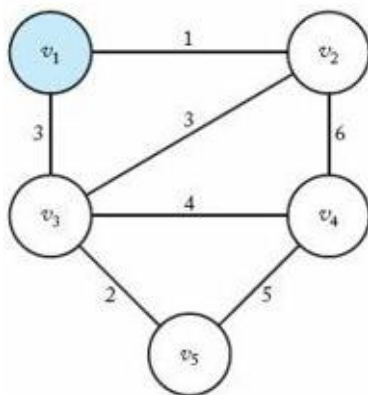
// 해답점검

 문제 해결

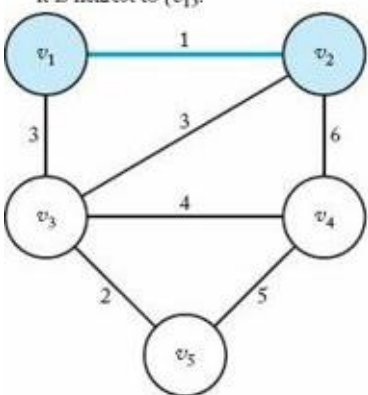
}

- vertex를 추가해도 cycle이 생기지 않기 때문에 적절성 검사는 불필요함

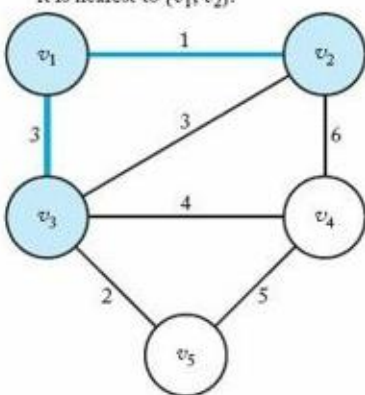
1. Vertex v_1 is selected first.



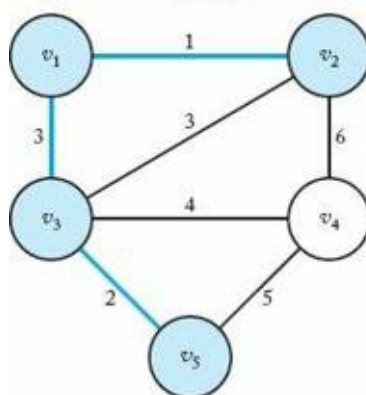
2. Vertex v_2 is selected because it is nearest to $\{v_1\}$.



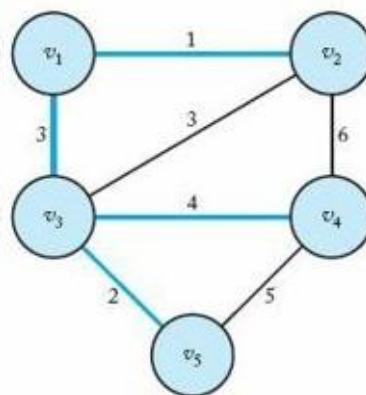
3. Vertex v_3 is selected because it is nearest to $\{v_1, v_2\}$.



4. Vertex v_3 is selected because it is nearest to $\{v_1, v_2, v_3\}$.



5. Vertex v_4 is selected.



■ 프림(Prim)의 알고리즘 (Minimum spanning tree algorithm)

■ Notation

- $W[i][j] = \begin{cases} \text{weight} (v_i \text{와 } v_j \text{를 연결하는 edge가 존재}) \\ \infty & (v_i \text{와 } v_j \text{를 연결하는 edge가 존재하지 않음}) \\ 0 & (i = j) \end{cases}$
- $\text{nearest}[i] = v_i$ 에서 가장 가까운 Y 에 속한 vertex의 인덱스
- $\text{distance}[i] = v_i$ 와 $\text{nearest}[i]$ 의 vertex를 연결하는 edge의 weight

■ Algorithm 4.1 설명

1) 초기화

- $Y = \{v_1\}$ 이므로, $\text{nearest}[i] = 1$
- $\text{distance}[i]$ 는 v_1 과 v_i 를 연결하는 edge의 weight

2) Y 와 가장 가까운 vertex를 찾는다

- min은 가장 가까운 distance
- v_{near} 는 그 때의 index

3) $\text{distance}[v_{\text{near}}] = -1$: 선택된 vertex를 제외시킨다

4) 확장된 vertex와 연결된 vertex의 정보($\text{distance}[i]$, $\text{nearest}[i]$) 업데이트

■ 프림(Prim)의 알고리즘 (Minimum spanning tree algorithm)

■ Algo 4.1 Prim's Algorithm

- 문제: 최소비용 신장트리를 구하시오
- 입력: n (vertex의 개수),
 W (connected, weighted undirected graph)
- 출력: F (최소비용 신장트리에 있는 edge의 집합)

```
void prim (int n, const number W[][],  
           set_of_edges& F) {  
    index i, vnear;  
    number min;  
    edge e;  
    index nearest [2..n];  
    number distance [2..n];  
    F =  $\emptyset$ ;  
    for (i= 2; i <= n; i++) {      // (1)  
        nearest[i] = 1;  
        distance[i] = W[1][i];  
    }
```

```
    repeat (n - 1 times) {  
        min =  $\infty$ ;  
        for (i= 2; i <= n; i++)    // (2)  
            if (0  $\leq$  distance[i] < min) {  
                min = distance[i];  
                vnear = i;  
            }  
        e = edge ( $v_{vnear}, v_{nearest[vnear]}$ )  
        add e to F;  
        distance [vnear] = -1;    // (3)  
        for (i= 2; i <= n; i++)    // (4)  
            if (W[i][vnear] < distance[i]) {  
                distance[i]=W[i][vnear];  
                nearest [i] = vnear;  
            }  
    }  
}
```

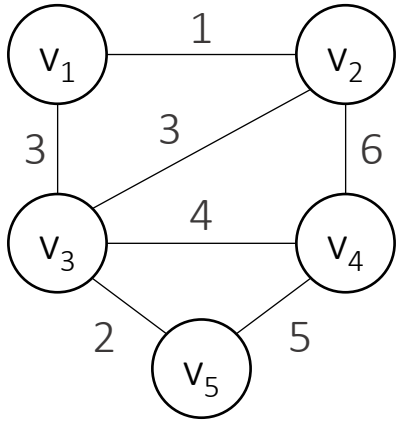
■ 프림(Prim)의 알고리즘 (Minimum spanning tree algorithm)

- Algo 4.1의 일정 시간 복잡도(Every-case time complexity) 분석
 - 단위 연산: repeat-루프 안에 있는 2개의 for-루프 안의 명령문
 - 입력 크기: 그래프의 vertex 개수 n

$$T(n) = 2(n-1)(n-1) \in \theta(n^2)$$

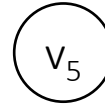
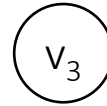
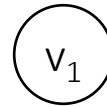
■ 크루스칼(Kruskal)의 알고리즘 (Minimum spanning tree algorithm)

■ 크루스칼 알고리즘의 진행절차

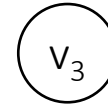
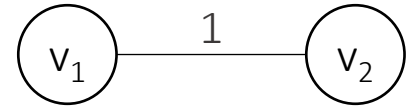
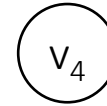
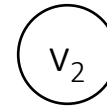


최소비용 신장트리 계산

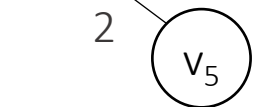
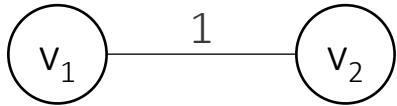
- (v1, v2) 1
- (v3, v5) 2
- (v1, v3) 3
- (v2, v3) 3
- (v3, v4) 4
- (v4, v5) 5
- (v2, v4) 6



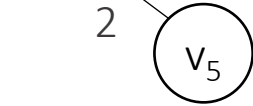
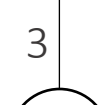
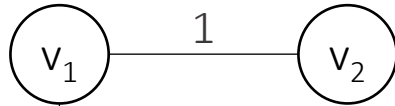
2. 서로소 부분집합 생성



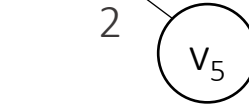
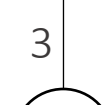
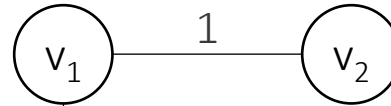
3. edge (v1, v2) 선택



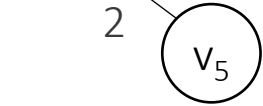
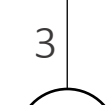
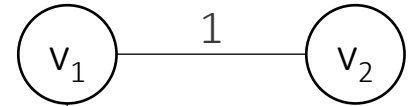
4. edge (v3, v5) 선택



5. edge (v1, v3) 선택



6. edge (v2, v3) 선택



7. edge (v3, v4) 선택

■ 크루스칼(Kruskal)의 알고리즘 (Minimum spanning tree algorithm)

■ (high-level) 알고리즘

```
F = ∅ // edge의 집합을 공집합으로 초기화
V의 서로소 부분집합 생성 // 각 vertex 하나씩 포함된 집합 생성
E에 속한 edge를 weight 순으로 오름차순 정렬
while (미해결) {
    다음 가장 작은 weight의 edge 선택 // 선택과정
    if (edge가 서로소 부분집합의 두 vertex를 연결) { // 적절성검사
        부분집합을 병합
        그 edge를 F에 추가
    }
    if (모든 부분집합이 하나로 병합) // 해답점검
        문제 해결
}
```

■ 알고리즘에서 사용되는 연산

- $\text{initial}(n)$: n 개의 서로소 부분집합을 초기화
- $p = \text{find}(i)$: p 가 인덱스 i 를 포함하는 집합을 가리킨다
- $\text{merge}(p, q)$: p 와 q 가 가리키고 있는 두 개의 집합을 하나로 병합
- $\text{equal}(p, q)$: p 와 q 가 같은 집합을 가리키면 true, 아니면 false

■ 서로소 집합의 데이터 구조 (appendix)

- 서로소 부분집합
 - 각 부분집합은 그래프의 각 vertex(node)를 하나씩만 포함
- 추상 데이터 구조(abstract data type)
 - 데이터 객체
 - 객체에 허용되는 연산

- 전체 영역 $U = \{A, B, C, D, E\}$ 를 가정

- U 의 멤버로부터 서로소 집합 생성
for (each $x \in U$)

 makeset(x);

실행결과: $\{A\}, \{B\}, \{C\}, \{D\}, \{E\}$

- 포인터 p 와 q 는 B 또는 C 를 포함하는 집합을 가리키는 포인터

set_pointer p, q ;

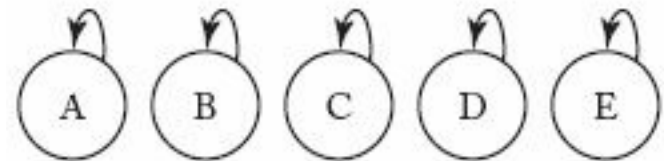
$p = \text{find}('B');$

$q = \text{find}('C');$

실행결과: $\{A\}, \{B\}, \{C\}, \{D\}, \{E\}$

 ↑ ↑
 p q

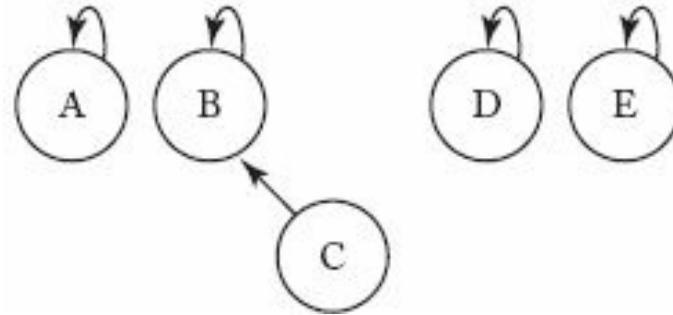
(a) Five disjoint sets represented by inverted trees.



■ 서로소 집합의 데이터 구조 (appendix)

- 두 개의 집합을 합병
 $p = \text{find}('B');$
 $q = \text{find}('C');$
 $\text{merge}(p, q)$
실행결과: $\{A\}, \{B, C\}, \{D\}, \{E\}$
- 두 집합이 같은지 검사
 $r = \text{find}('A');$
 $\text{equal}(p, q) \quad // \text{true}$
 $\text{equal}(p, r) \quad // \text{false}$

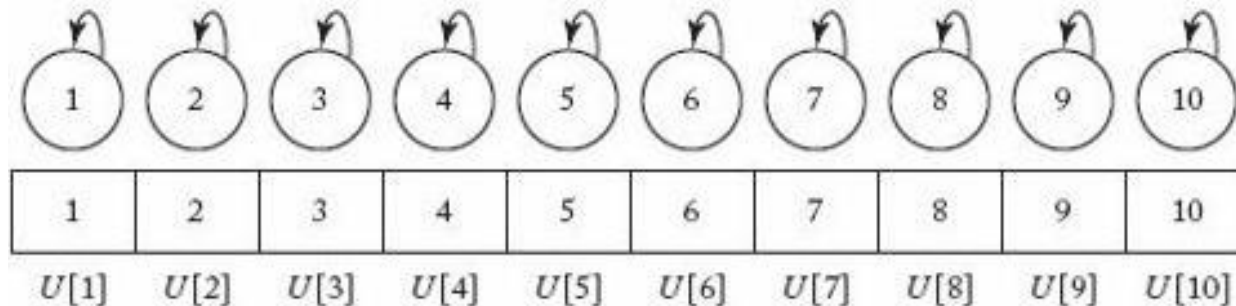
(b) The inverted trees after [B] and [C] are merged.



■ 역 포인터를 가진 트리

- 각 node의 포인터는 parent를 가리킴
- Root node의 포인터는 자신을 가리킴
- 배열 U 를 사용하여 트리 구현

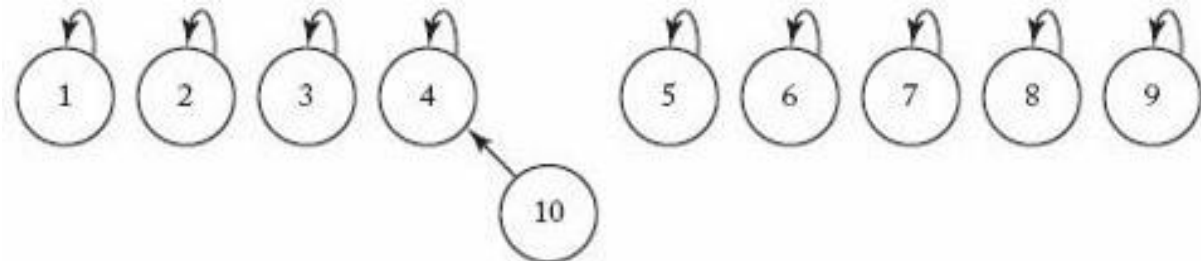
$U[i] = i$



서로소 집합의 데이터 구조 (appendix)

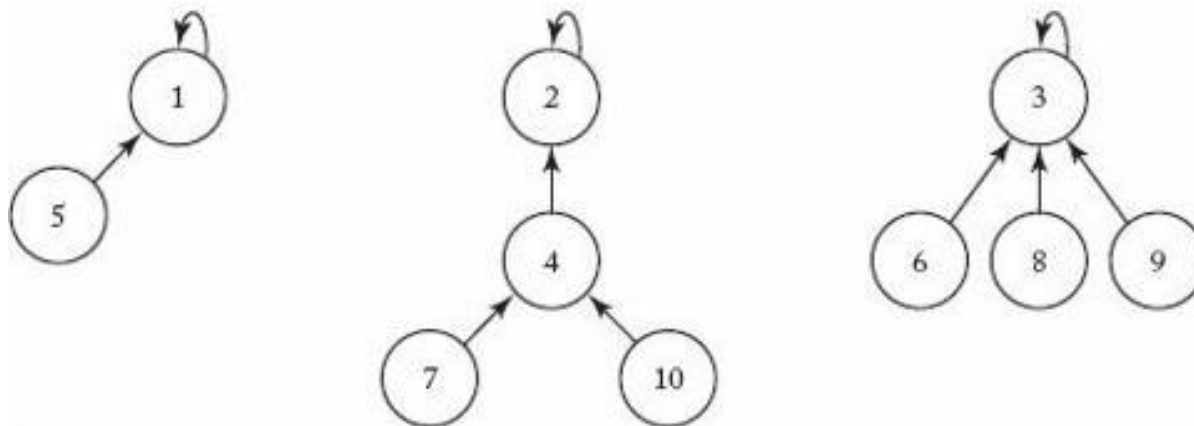
- {4}와 {10}의 합병

$$U[10] = 4$$



1	2	3	4	5	6	7	8	9	4
$U[1]$	$U[2]$	$U[3]$	$U[4]$	$U[5]$	$U[6]$	$U[7]$	$U[8]$	$U[9]$	$U[10]$

- 여러 번 합병 후 역 트리와 배열
 - 합병의 순서는 트리의 구조를 결정



1	2	3	2	1	3	4	3	3	4
$U[1]$	$U[2]$	$U[3]$	$U[4]$	$U[5]$	$U[6]$	$U[7]$	$U[8]$	$U[9]$	$U[10]$

■ 서로소 집합의 데이터 구조 (appendix)

■ 서로소 집합 데이터구조 1

const int n = 전체집합이 가지고
있는 요소의 개수;

typedef int index;

typedef index set_pointer;

typedef index universe [1..n];

universe U;

void makeset (index i) {

 U[i] = i;

}

set_pointer find (index i) {

 index j;

 j = i;

 while (U[j] != j)

 j = U[j];

 return j;

}

void merge (set_pointer p, set_pointer q) {

 if (p < q) // p는 합병된 집합을 가리키는 포인터

 U[q] = p; // q는 집합을 가리키는 포인터가 아님

 else

 U[p] = q; // q는 합병된 집합을 가리키는 포인터

 // p는 집합을 가리키는 포인터가 아님

}

bool equal (set_pointer p, set_pointer q) {

 if (p == q)

 return true;

 else

 return false;

}

void initial (int n) {

 index i;

 for (i=1; i <= n; i++)

 makeset (i);

}

■ 서로소 집합의 데이터 구조 (appendix)

■ 시간복잡도

- n 개의 서로소 집합 초기화, m 번 루프 실행
- $\text{initial}()$ 의 시간복잡도는 $\theta(n)$
- $\text{equal}()$, $\text{merge}()$ 는 상수시간 복잡도 $\theta(1)$

■ $\text{find}()$ 의 시간 복잡도

- 최악의 경우 실행되는 비교 횟수 (단위연산)

- $\text{find}(6)$ 의 비교 횟수

$\text{merge}(\{5\}, \{6\})$; 2 번

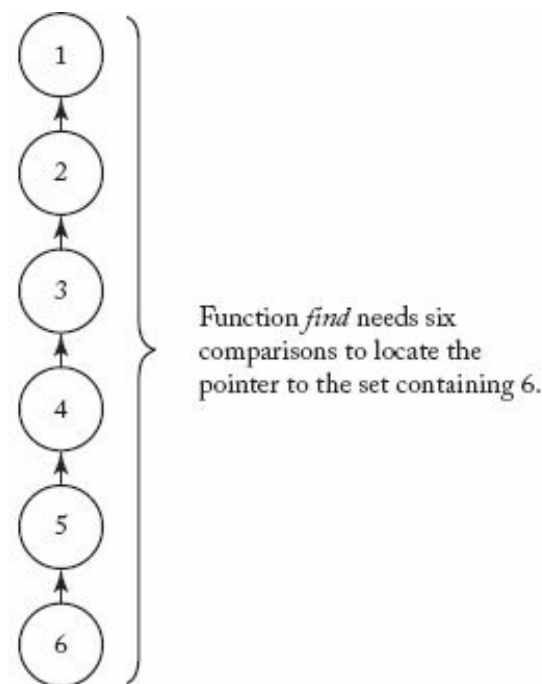
$\text{merge}(\{4\}, \{5, 6\})$; 3 번

$\text{merge}(\{3\}, \{4, 5, 6\})$; 4 번

$\text{merge}(\{2\}, \{3, 4, 5, 6\})$; 5 번

$\text{merge}(\{1\}, \{2, 3, 4, 5, 6\})$; 6 번

- 각 합병(merge) 이후 $\text{find}(6)$ 을 위한 총 비교 횟수
 $2+3+ \dots +(m+1) \in \theta(m^2)$

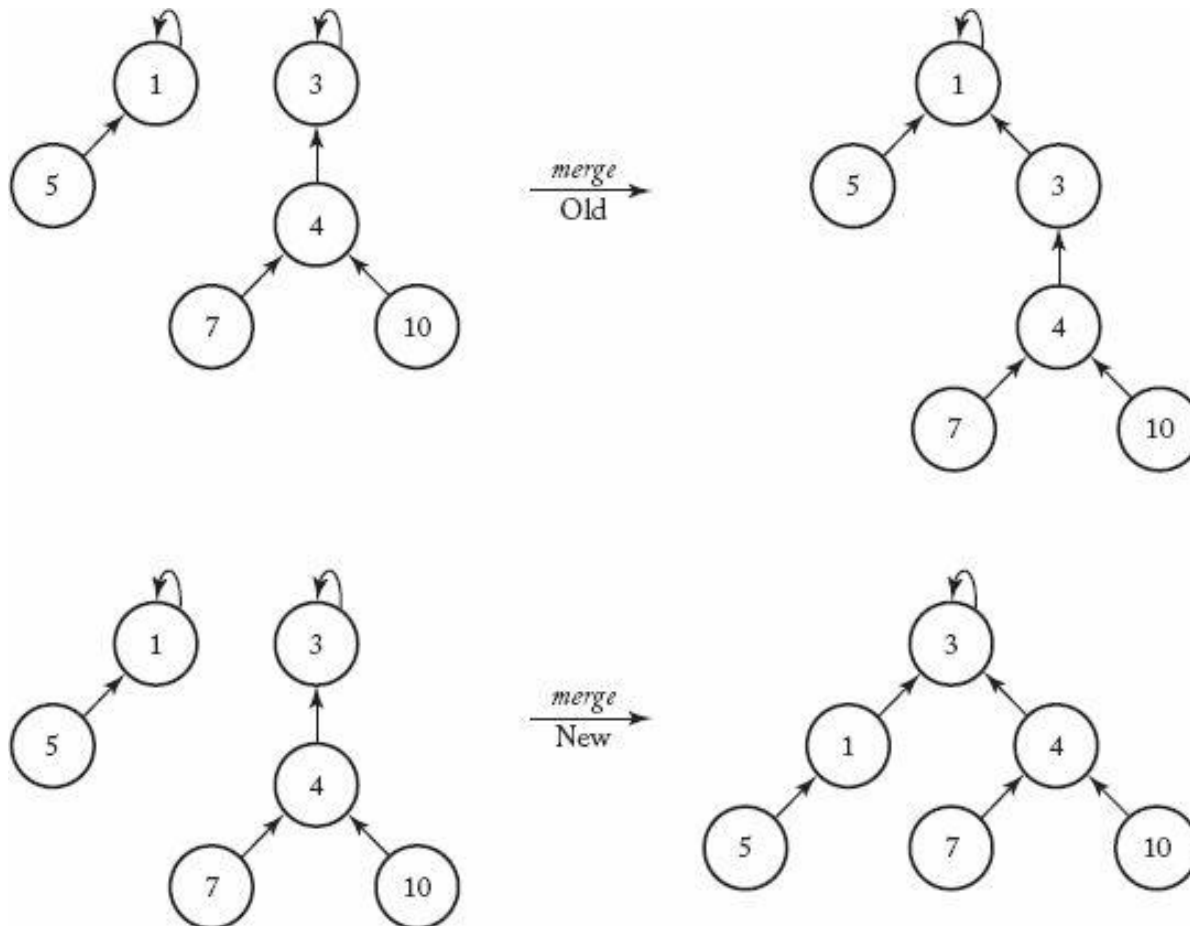


1	1	2	3	4	5
$U[1]$	$U[2]$	$U[3]$	$U[4]$	$U[5]$	$U[6]$

서로소 집합의 데이터 구조 (appendix)

서로소 집합 데이터구조 2

- 각 root node에 트리의 깊이를 저장
- 트리를 합병할 때 깊이가 작은 트리를 child node로 합병



■ 서로소 집합의 데이터 구조 (appendix)

■ 서로소 집합 데이터구조 2

const int n = 전체 집합이 가지고
있는 요소의 개수;

typedef int index;

typedef index set_pointer;

struct nodetype {

index parent;

int depth;

}

typedef nodetype universe [1..n];

universe U;

void makeset (index i); {

U[i].parent = i;

U[i].depth = 0;

}

set_pointer find (index i) {

index j;

j = i;

while (U[j].parent != j)

j = U[j].parent;

return j

}

void merge (set pointer p, set pointer q) {

if (U[p].depth == U[q].depth) {

U[p].depth = U[p].depth + 1; // 트리의 깊이는 증가

U[q].parent = p;

}

else (U[p].depth < U[q].depth) // 깊이가 작은 트리를

U[p].parent = q; // child node로 만든다

else

U[q].parent = p;

}

bool equal (set_pointer p, set pointer q) {

if (p == q)

return true;

else

return false;

}

void initial (int n) {

index i;

for (i=1; i <= n; i++)

makeset(i);

}

equal, find, merge를
호출하는 횟수가 상수인
루프를 m번 반복하는
최악 시간 복잡도는
 $\theta(m \lg m)$

■ 서로소 집합의 데이터 구조 (appendix)

- 서로소 집합 데이터구조 3
 - 가장 작은 멤버를 root node에 저장

```
const int n = 전체집합이 가지고  
              있는 요소의 개수;
```

```
typedef int index;
```

```
typedef index set_pointer;
```

```
struct nodetype {
```

```
    index parent;
```

```
    int depth;
```

```
    int smallest;
```

```
};
```

```
typedef nodetype universe [1..n];
```

```
universe U;
```

```
void makeset (index i) {
```

```
    U[i].parent = i;
```

```
    U[i].depth=0;
```

```
    U[i].smallest = i;
```

```
}
```

```
int small (set_pointer p) {
```

```
    return U[p]. smallest;
```

```
}
```

```
void merge (set_pointer p, set_pointer q) {
```

```
    if (U[p].depth == U[q].depth) {
```

```
        U[p].depth = U[q].depth + 1;
```

```
// 트리의 깊이는 증가
```

```
        U[q].parent = p;
```

```
        if (U[q]. smallest < U[p]. smallest)
```

```
            U[p]. smallest = U[q]. smallest;
```

```
// 가장 작은 인덱스는
```

```
    }
```

```
// q의 트리
```

```
    else if (U[p]. depth < U[q]. depth) {
```

```
// 깊이가 작은 트리를
```

```
        U[p]. parent = q;
```

```
// child node로 만든다
```

```
        if (U[p]. smallest < U[q]. smallest)
```

```
            U[q]. smallest = U[p]. smallest;
```

```
// 가장 작은 인덱스는
```

```
    }
```

```
// p의 트리
```

```
    else {
```

```
        U[q]. parent = p;
```

```
        if (U[q]. smallest < U[p]. smallest)
```

```
            U[p]. smallest = U[q]. smallest;
```

```
// 가장 작은 인덱스는
```

```
    }
```

```
// q의 트리
```

```
}
```

■ 크루스칼(Kruskal)의 알고리즘 (Minimum spanning tree algorithm)

■ Algo 4.2 Kruskal's Algorithm

- 문제: 최소비용 신장트리를 구하시오
- 입력: n (vertex의 개수), m (edge의 개수)
E (connected, weighted undirected graph의 edge와 weight)
- 출력: F (최소비용 신장트리에 있는 edge의 집합)

```
void Kruskal (int n, int m, set_of_edges E, set_of_edges& F) {  
    index i, j ;  
    set_pointer p, q ;  
    edge e ;  
    E에 있는 m개의 edge를 weight순으로 오름차순 정렬 ;  
    F =  $\emptyset$  ;  
    initial(n) ;  
    while (F에 있는 edge의 수가 n-1보다 작다) {  
        e = 고려하지 않은 edge 중 가중치가 최소인 edge ;  
        i, j = e로 연결된 vertex의 인덱스 ;  
        p = find(i), q = find(j) ;  
        if (! equal(p, q)) {  
            merge(p, q) ;  
            e를 F에 추가 ;  
        } } }
```

■ 크루스칼(Kruskal)의 알고리즘 (Minimum spanning tree algorithm)

- Algo 4.2의 최악 시간 복잡도(Worst-case time complexity) 분석
 - 단위 연산: 비교 명령문
 - 입력 크기: n (vertex의 개수), m (edge의 개수)
 - edge의 정렬 시간 (합병 정렬 Algo 2.2)
$$W(m) \in \theta(m \lg m)$$
 - initial() : n 개의 서로소 집합을 초기화하는데 걸리는 시간
$$T(n) \in \theta(n)$$
 - while 루프에서 걸리는 시간
 - 최악의 경우 모든 edge를 고려함 (루프 m 번 반복)
 - (서로소 집합 데이터구조 2) find, equal, merge를 호출하는 횟수가 상수인 루프를 m 번 반복하는 시간 복잡도는
$$W(m) \in \theta(m \lg m)$$
 - $m \geq n-1$ 이므로, 서로소 집합을 정렬하고 조작하는 시간이 전체 시간을 차지한다. 그러므로,
$$W(m) \in \theta(m \lg m)$$
 - 최악의 경우 모든 vertex는 연결되므로, $m = \frac{1}{2}n(n-1) \in \theta(n^2)$
$$W(m, n) \in \theta(n^2 \lg n^2) = \theta(n^2 2 \lg n) = \theta(n^2 \lg n)$$

■ Prim 알고리즘과 Kruskal 알고리즘의 비교

■ Time complexity

- Prim 알고리즘: $T(n) \in \theta(n^2)$
- Kruskal 알고리즘: $W(m, n) \in \theta(m \lg m)$, $W(m, n) \in \theta(n^2 \lg n)$
- vertex의 개수 n , edge의 개수 m 인 연결된 그래프에서

$$n - 1 \leq m \leq \frac{1}{2}n(n-1)$$

- Kruskal 알고리즘의 time complexity는

저밀도(sparse) 그래프에서 $W(m, n) \in \theta(n \lg n) < \theta(n^2)$

고밀도(dense) 그래프에서 $W(m, n) \in \theta(n^2 \lg n) > \theta(n^2)$

■ Discussion

- 1930년 Jarnik이 Prim 알고리즘 개발
- 1956년 Kruskal이 Kruskal 알고리즘 개발
- 1977년 Johnson이 힙(heap) 데이터 구조를 사용하여 $\theta(m \lg n)$ Prim 알고리즘 구현
- 1987년 Fredman, Tarjan이 피보나치 힙 데이터 구조를 사용하여 가장 빠른 Prim 알고리즘 구현 $\theta(m + n \lg n)$
 - 저밀도(sparse) 그래프에서 $W(m, n) \in \theta(n \lg n)$
 - 고밀도(dense) 그래프에서 $W(m, n) \in \theta(n^2)$

■ 단일출발점 최단경로 알고리즘 (Dijkstra)

■ (Algo 3.3-3.5) 최단 경로 알고리즘

- vertex가 n 개인 weighted, directed graph에서, 각 vertex에서 다른 모든 vertex로 가는 최단 경로를 계산하는 $\theta(n^3)$ 알고리즘

■ 하나의 vertex에서 다른 모든 vertex로 가는 최단 경로 (high-level) 알고리즘

$Y = \{v_1\}$

$F = \emptyset$

while (미해결) {

v_1 에서 Y 에 속한 vertex만 거쳐서 가는 최단 경로를
가진 vertex v 를 $V-Y$ 에서 선택

// 선택과정 & 적절성검사

새로운 vertex v 를 Y 에 추가

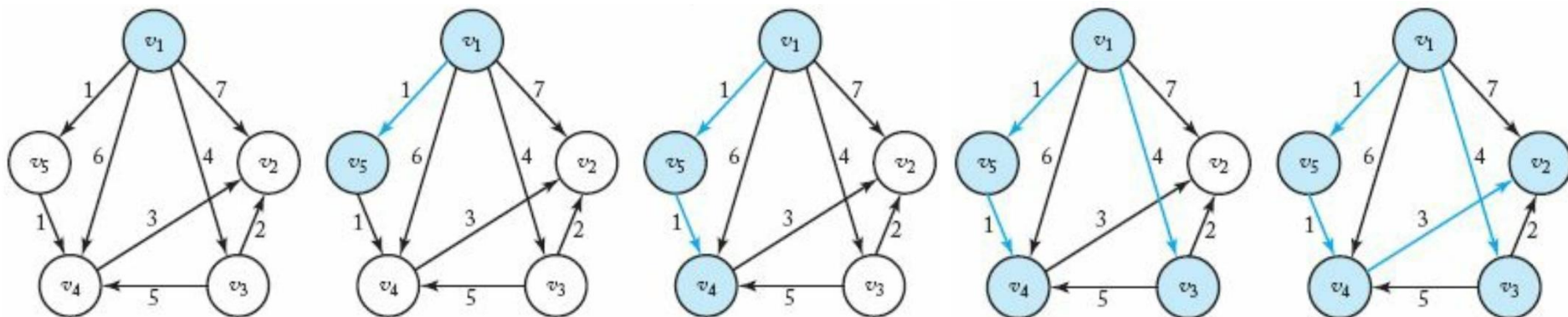
vertex v 로 가는 edge를 F 에 추가

if ($Y == V$)

// 해답점검

문제 해결

}



■ 단일출발점 최단경로 알고리즘 (Dijkstra)

■ Notation

- $W[i][j] = \begin{cases} \text{weight} (v_i \text{에서 } v_j \text{로 가는 edge가 존재}) \\ \infty & (v_i \text{에서 } v_j \text{로 가는 edge가 존재하지 않음}) \\ 0 & (i = j) \end{cases}$
- $\text{touch}[i] = Y$ 에 속한 vertex만 거쳐서 v_1 에서 v_i 로 가는 최단 경로의 가장 마지막 edge가 $\langle v, v_i \rangle$ 라고 할 때, Y 에 속한 vertex v 의 인덱스
- $\text{length}[i] = Y$ 에 속한 vertex만 거쳐서 v_1 에서 v_i 로 가는 최단 경로의 길이

■ Algorithm 4.3 설명

1) 초기화

- $Y = \{v_1\}$ 이므로, $\text{touch}[i] = 1$
- $\text{length}[i]$ 는 v_1 에서 v_i 로 가는 edge의 weight

2) 최단 경로를 갖는 vertex를 찾는다

- min은 최단 경로의 length
- v_{near} 는 최단 경로의 vertex v_i 의 인덱스

3) 확장된 vertex의 정보($\text{touch}[i]$, $\text{length}[i]$) 업데이트

4) $\text{length}[v_{\text{near}}] = -1$: 선택된 vertex를 제외시킨다

■ 단일출발점 최단경로 알고리즘 (Dijkstra)

■ Algo 4.3 Dijkstra's Algorithm

- 문제: v_1 에서 다른 모든 vertex로 가는 최단경로를 구하시오.
- 입력: n (vertex의 개수),
 W (connected, weighted directed graph)
- 출력: F (최단경로에 있는 edge의 집합)

```
void dijkstra (int n, const number W[][],  
               set_of_edges& F) {  
    index i, vnear;  
    edge e;  
    index touch [2..n];  
    number length [2..n];  
    F =  $\emptyset$ ;  
    for (i= 2; i <= n; i++) {      // (1)  
        touch[i] = 1;  
        length[i] = W[1][i];  
    }
```

```
    repeat (n - 1 times) {  
        min =  $\infty$ ;  
        for (i= 2; i <= n; i++)    // (2)  
            if ( $0 \leq \text{length}[i] < \text{min}$ ) {  
                min = length[i];  
                vnear = i;  
            }  
        e = edge ( $v_{\text{touch}[\text{vnear}]}$ ,  $v_{\text{vnear}}$ )  
        add e to F;  
        for (i= 2; i <= n; i++)    // (3)  
            if ( $\text{length}[\text{vnear}] + W[\text{vnear}][i] < \text{length}[i]$ ) {  
                length[i] = length[vnear] + W[vnear][i];  
                touch[i] = vnear;  
            }  
        length [vnear] = -1;      // (4)  
    }
```

■ 단일출발점 최단경로 알고리즘 (Dijkstra)

- Algo 4.3의 일정 시간 복잡도(Every-case time complexity) 분석
 - 단위 연산: repeat-루프 안에 있는 2개의 for-루프 안의 명령문
 - 입력 크기: 그래프의 vertex 개수 n
$$T(n) = 2(n-1)(n-1) \in \theta(n^2)$$

■ 허프만 코드 (Huffman code)

■ 효율적인 파일 저장 방법

- 데이터는 이진 코드(binary code)로 표현하고, 각 문자는 코드워드(code word)라고 하는 유일한 이진 문자열로 표현한다
- 길이가 고정된 이진 코드(fixed-length binary code)
 - 각 문자를 표현하는 비트의 개수가 일정
 - 문자 집합이 {a, b, c} 경우, 2비트만으로 코드화 가능 → a: 00, b: 01, c: 11
 - ababcbbbc → 000100011101010111 (18 bits)
- 길이가 변하는 이진 코드(variable-length binary code)
 - 각 문자마다 표현하는 비트의 개수가 다름
 - “ababcbbbc”에서 ‘b’가 가장 빈번하게 나오므로, ‘b’를 0으로 코드화 → a: 10, b: 0, c: 11
 - ababcbbbc → 1001001100011 (13 bits)

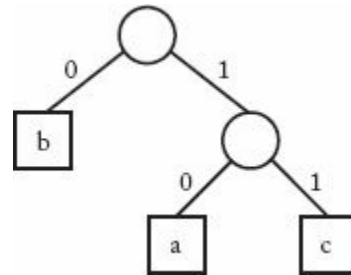
■ 최적 이진코드(optimal binary code) 문제

- 주어진 파일에 있는 문자들을 이진코드로 표현하는데 필요한 비트의 개수가 최소가 되는 이진문자코드를 찾는 것

■ 허프만 코드 (Huffman code)

- 전치 코드 (prefix code)
 - 길이가 변하는 이진 코드(variable-length binary code)의 한 종류
 - 한 문자의 코드워드가 다른 문자의 코드워드의 앞부분이 될 수 없다
(ex. a: 10, b: 0, c: 11, 길이가 고정된 코드)
 - Leaf가 코드문자로 구성된 binary tree로 표현 가능

a: 10, b: 0, c: 11



- 예제 4.7
 - 파일을 코드화 하였을 때, 필요한 비트의 수 계산

$$bits(T) = \sum_{i=1}^n frequency(v_i) depth(v_i)$$

- $\{v_1, v_2, \dots, v_n\}$: 파일에 있는 문자의 집합
- $Frequency(v_i)$: v_i 가 파일에 나타나는 횟수
- $Depth(v_i)$: 이진 트리 T 에서 v_i 의 깊이

■ 허프만 코드 (Huffman code)

■ 예제 4.7

- 파일에 있는 문자의 빈도수
- 파일 코드화 방법 C1, C2, C3

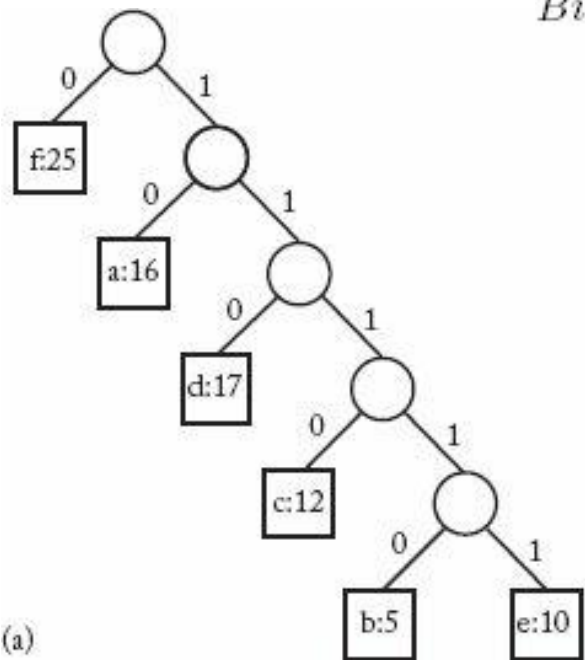
Character	Frequency	C1 (Fixed-Length)	C2	C3 (Huffman)
a	16	000	10	00
b	5	001	11110	1110
c	12	010	1110	110
d	17	011	110	01
e	10	100	11111	1111
f	25	101	0	10

- 소요된 비트수 계산
- C3가 최적

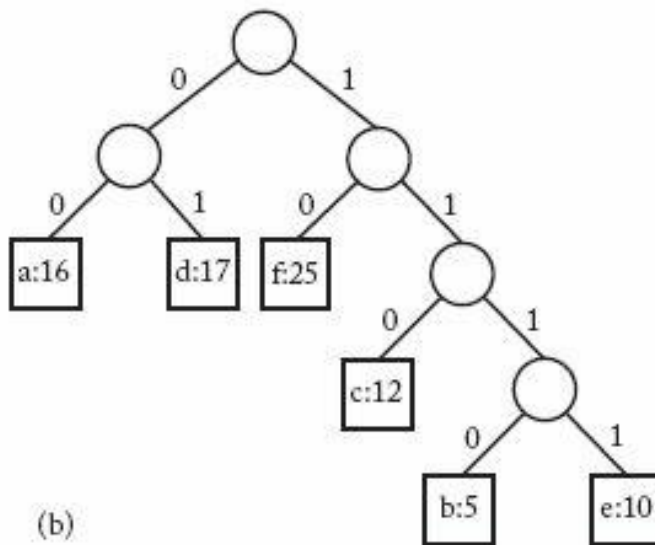
$$\text{Bits}(C1) = 16(3) + 5(3) + 12(3) + 17(3) + 10(3) + 25(3) = 255$$

$$\text{Bits}(C2) = 16(2) + 5(5) + 12(4) + 17(3) + 10(5) + 25(1) = 231$$

$$\text{Bits}(C3) = 16(2) + 5(4) + 12(3) + 17(2) + 10(4) + 25(2) = 212.$$



(a)



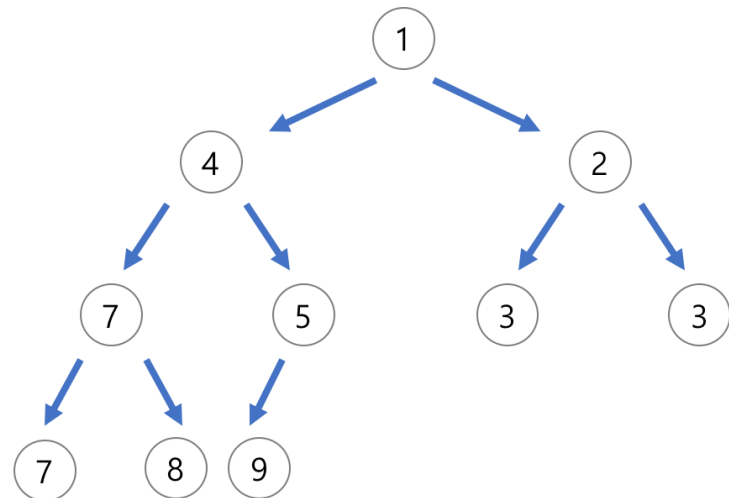
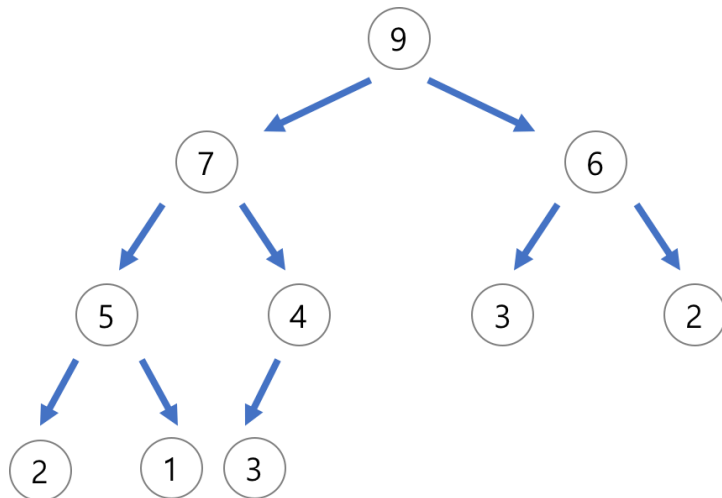
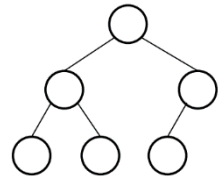
(b)

(a) C2 코드
(b) C3 허프만 코드

■ 힙 (heap)

■ Heap (자료구조)

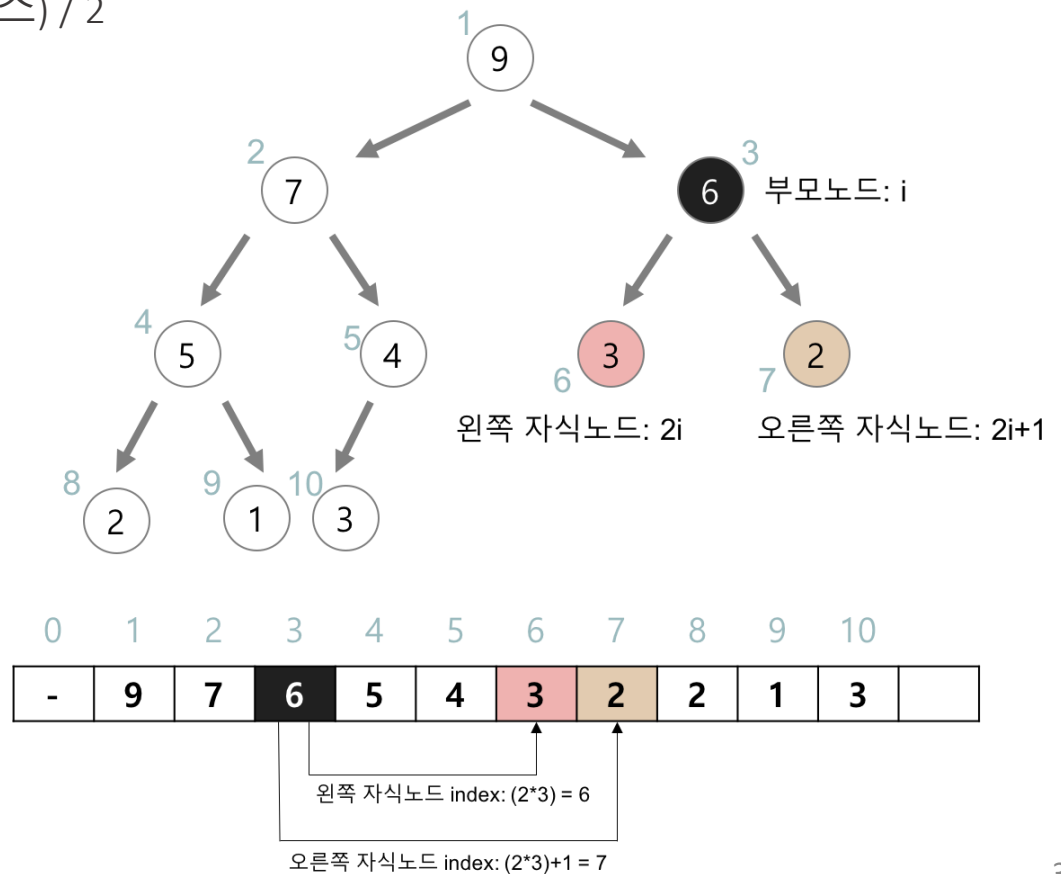
- 완전 이진 트리(complete binary tree)
 - 이진 트리(binary tree) 중에서 마지막 레벨을 제외한 모든 노드들이 채워진 트리
 - 마지막 레벨에서는 노드들이 왼쪽부터 오른쪽 방향으로 채워진 트리
- 힙 트리에서는 중복된 값을 허용
 - 이진 탐색 트리에서는 중복된 값을 허용하지 않는다
- 최대 힙(max heap)
 - $\text{key}(\text{부모 노드}) \geq \text{key}(\text{자식 노드})$
 - 루트에는 가장 큰 값이 저장
- 최소 힙(min heap)
 - $\text{key}(\text{부모 노드}) \leq \text{key}(\text{자식 노드})$
 - 루트에는 가장 작은 값이 저장



■ 힙 (heap)

■ (배열을 이용한) 힙의 구현

- 힙에서의 부모 노드와 자식 노드의 관계
 - 왼쪽 자식의 인덱스 = (부모의 인덱스) * 2
 - 오른쪽 자식의 인덱스 = (부모의 인덱스) * 2 + 1
 - 부모의 인덱스 = (자식의 인덱스) / 2

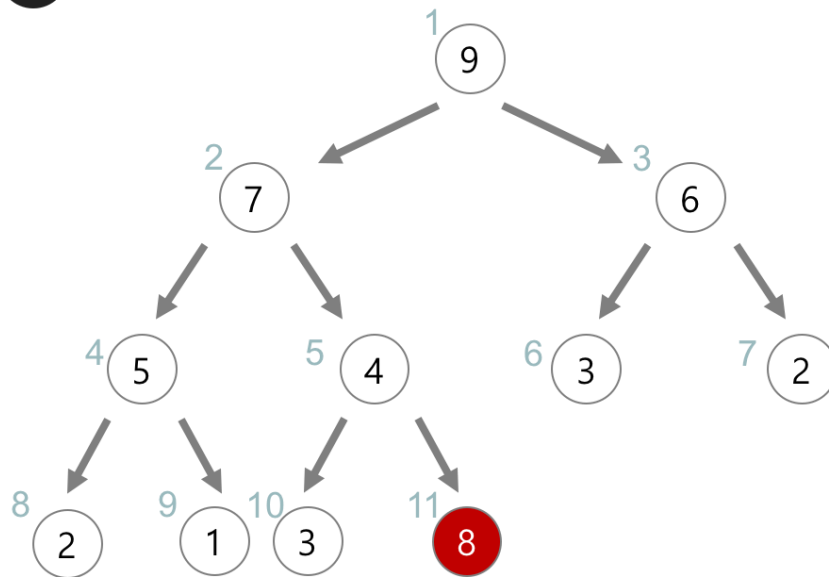


■ 힙 (heap)

■ 삽입 연산

- 힙에 새로운 값이 추가되면, 새로운 노드를 힙의 마지막 위치에 삽입
- 새로운 노드를 부모 노드들과 교환함으로써 힙의 성질을 만족
- 최악 시간 복잡도(Worst-case time complexity): $O(\lg n)$
- (ex) 최대 힙(max heap)에 새로운 값 8을 삽입

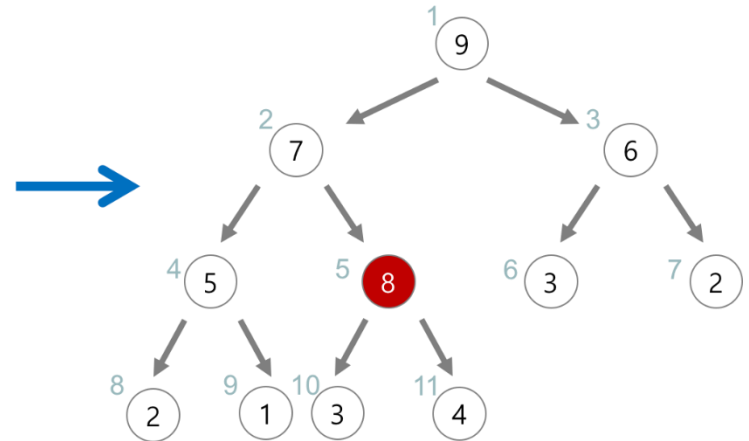
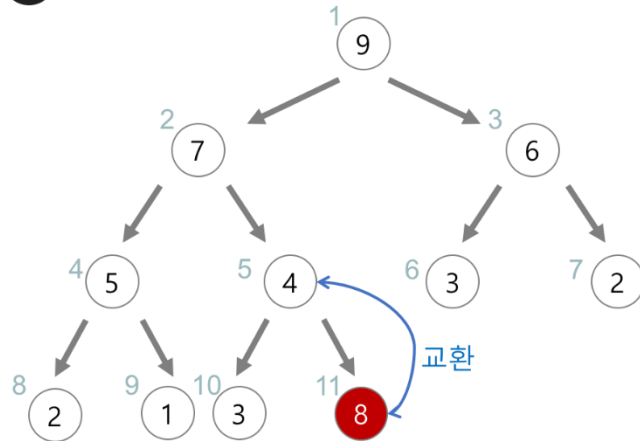
1 인덱스순으로 가장 마지막 위치에 이어서 새로운 요소 8을 삽입



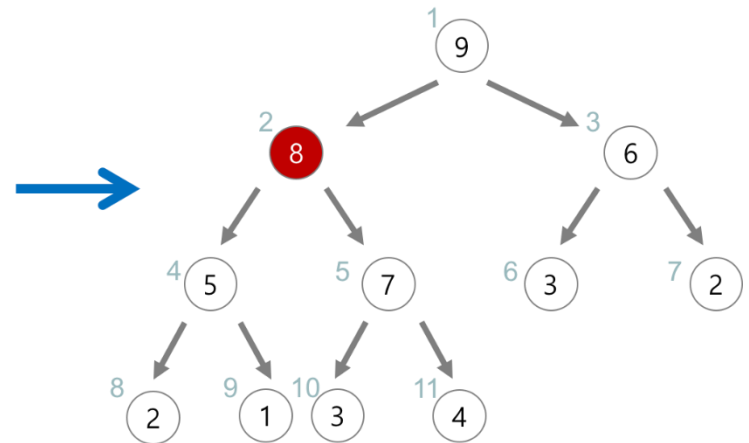
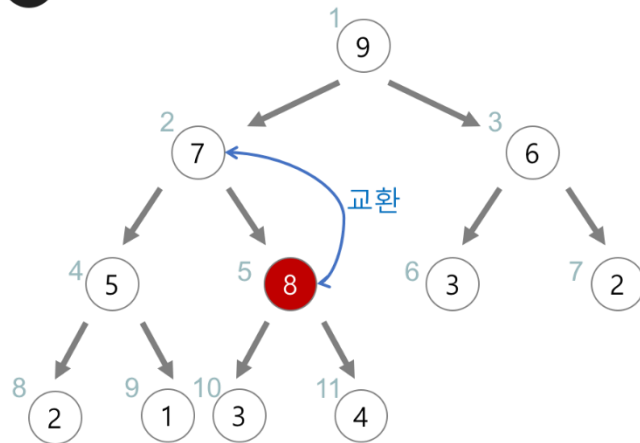
■ 힙 (heap)

■ 삽입 연산

2 부모 노드 4 < 삽입 노드 8 이므로 서로 교환



3 부모 노드 7 < 삽입 노드 8 이므로 서로 교환



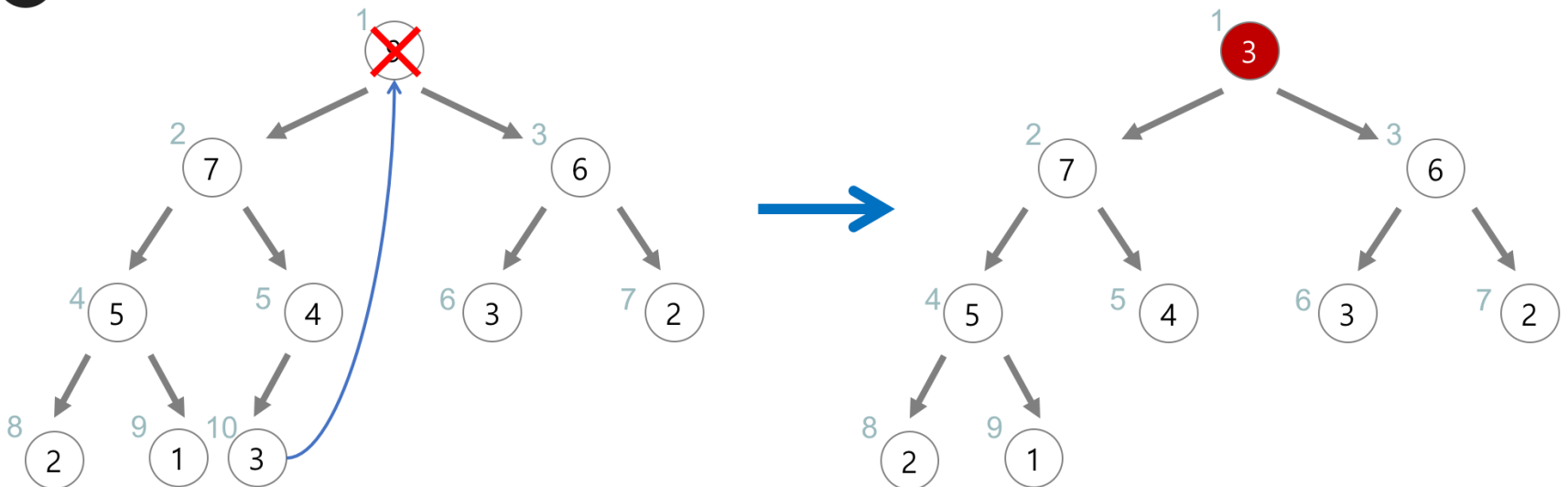
4 부모 노드 9 > 삽입 노드 8 이므로 더 이상 교환하지 않는다.

■ 힙 (heap)

■ 삭제 연산

- 힙에서 최대값/최소값(루트 노드) 삭제
 - 최대/최소(max/min) 힙에서 루트 노트는 최대값/최소값
- 힙의 마지막 노드를 삭제된 루트 노드로 이동
- 루트 노드부터 자식 노드들과 교환함으로써 힙의 성질을 만족
- 최악 시간 복잡도(Worst-case time complexity): $O(\lg n)$
- (ex) 최대 힙(max heap)에서 최대값 삭제

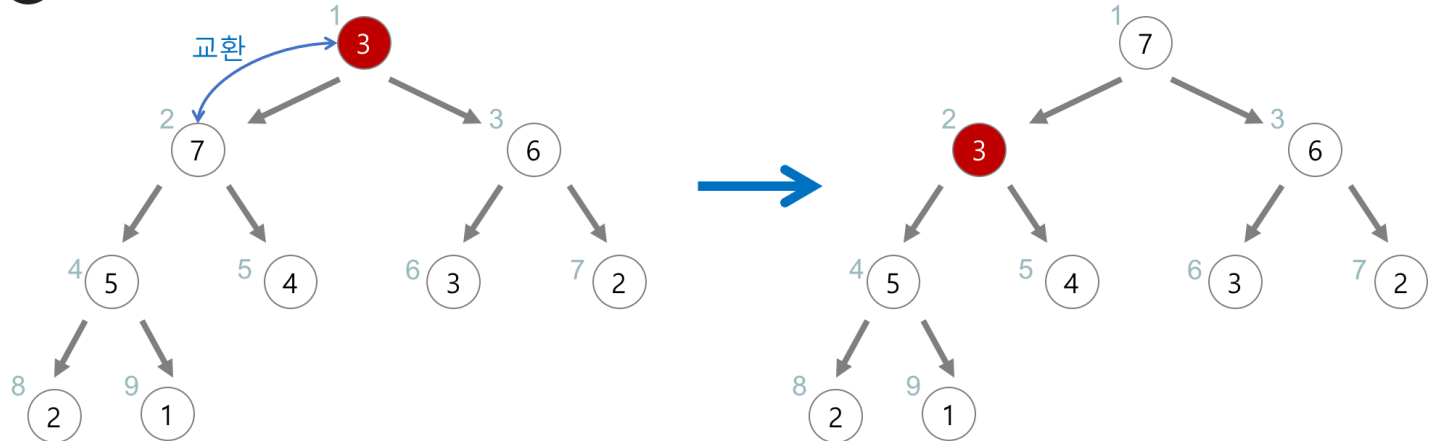
1 최댓값인 루트 노드 9를 삭제. (빈자리에는 최대 힙의 마지막 노드를 가져온다.)



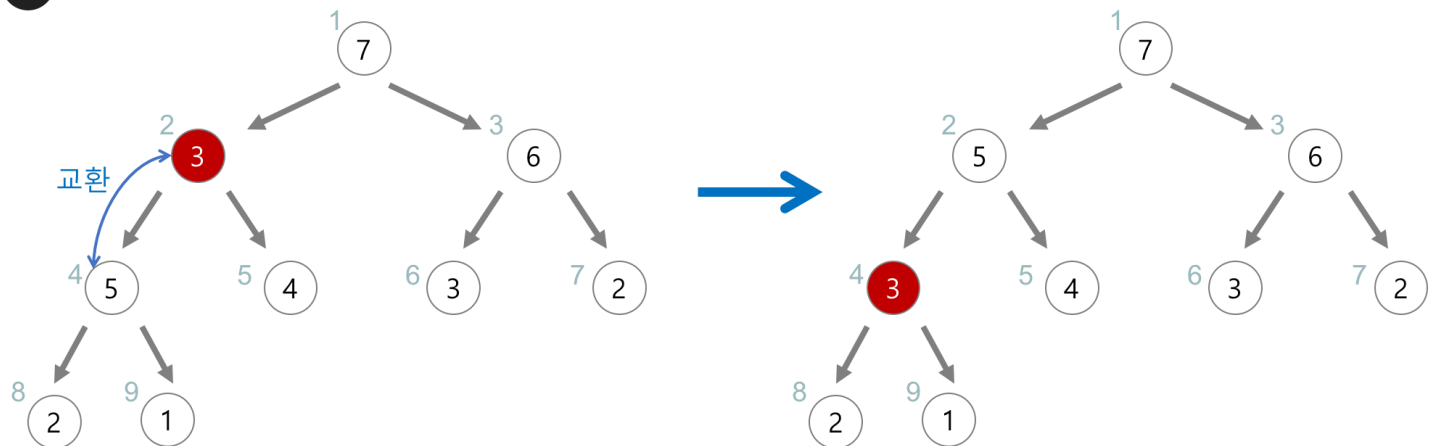
■ 힙 (heap)

■ 삭제 연산

- 2 삽입 노드와 자식 노드를 비교. 자식 노드 중 더 큰 값과 교환. (자식 노드 7 > 삽입 노드 3 이므로 서로 교환)



- 3 삽입 노드와 더 큰 값의 자식 노드를 비교. 자식 노드 5 > 삽입 노드 3 이므로 서로 교환



- 4 자식 노드 1, 2 < 삽입 노드 3 이므로 더 이상 교환하지 않는다.

■ 허프만 코드 (Huffman code)

■ 허프만 알고리즘

```
struct nodetype {  
    char symbol ;           // 문자값  
    int frequency ;        // 문자의 빈도수  
    nodetype* left ;  
    nodetype* right ;  
}
```

- 힙(heap)으로 구현된 우선순위 큐 (priority queue)를 사용
 - 우선순위 큐에서는 우선순위가 가장 높은 요소가 먼저 출력됨
 - 빈도수가 가장 낮은 문자가 먼저 출력됨
- 최악 시간 복잡도(Worst-case time complexity): $O(n \lg n)$
 - 힙 연산 1번의 최악 시간 복잡도: $O(\lg n)$

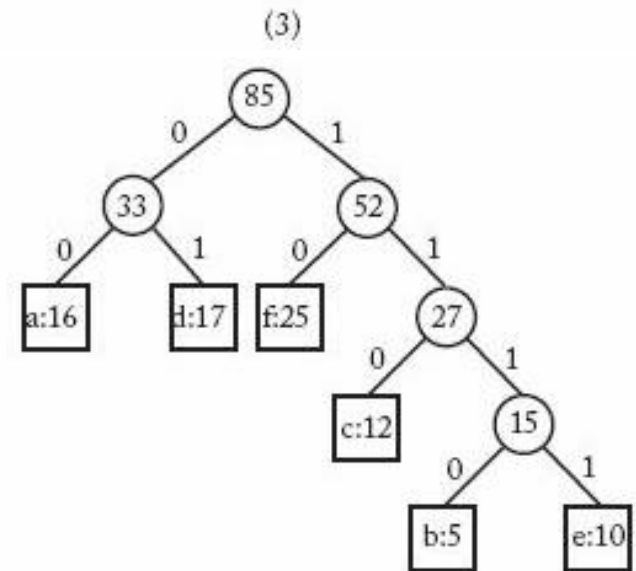
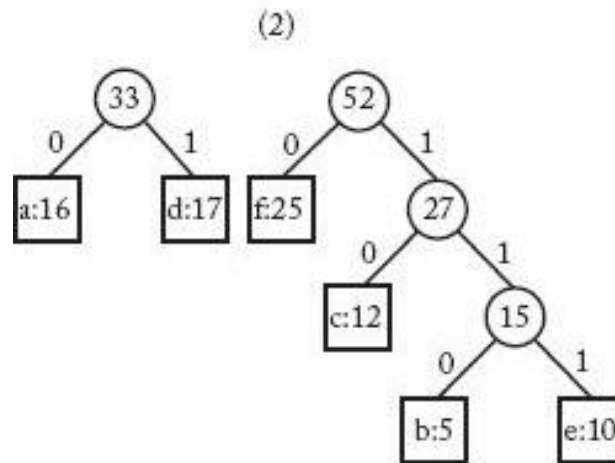
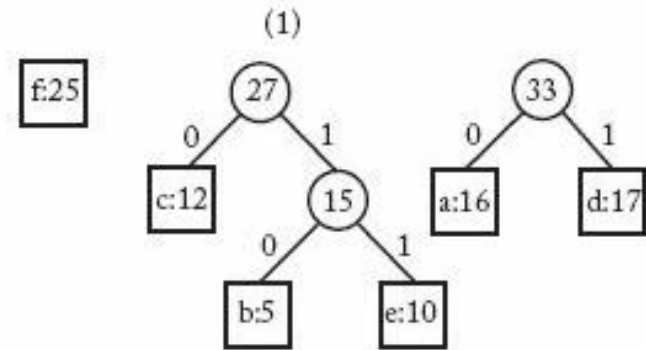
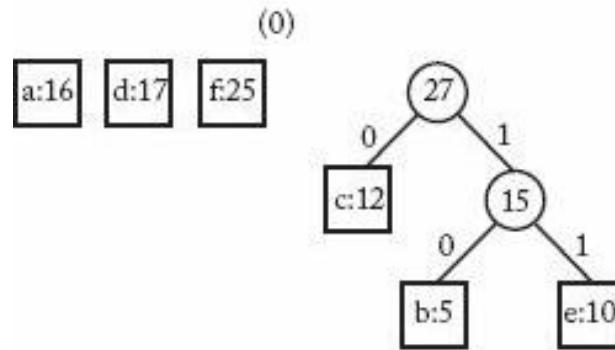
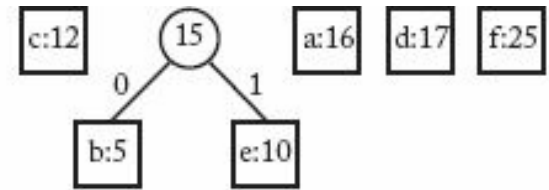
■ 허프만 코드 (Huffman code)

■ (high-level) 허프만 알고리즘

- n = 파일에서 문자의 개수
- 우선순위 큐 PQ에서 nodetype 레코드를 갖는 포인터(p) n 개를 배열
p -> symbol = 문자 ;
p -> frequency = 문자의 빈도수 ;
p -> left = p -> right = NULL ;
- 낮은 빈도수의 문자가 높은 우선순위를 갖는다
for (i=0 ; i <= n-1 ; i++) {
 // 해답 점검 과정 없음, (n-1) 번 반복 후 해답 출력
 remove(PQ, p) ;
 // 선택 과정
 remove(PQ, q) ;
 // 선택 과정
 r = new nodetype ;
 // 적절성 검사 과정 없음
 r -> left = p ; r -> right = q ;
 r -> frequency = p -> frequency + q -> frequency ;
 insert(PQ, r) ;
}
remove(PQ, r) ;
return r ;

■ 허프만 코드 (Huffman code)

b:5 e:10 c:12 a:16 d:17 f:25



- 예제 4.7에 대한
허프만 알고리즘